

Introduction to Industrial Control Systems

Digital Control, PLCs, Networks, SCADA & Intelligent Systems

Amit Kumar Singh
Assistant Professor (ECE)
School of Naval Architecture and Ocean Engineering
Indian Maritime University Visakhapatnam Campus, Sabbavaram Mandal
Visakhapatnam, Andhra Pradesh – 531 035

amitkumarsingh@imu.ac.in

September 25, 2026

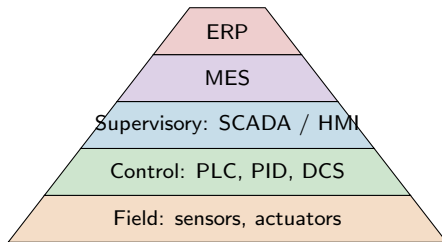
- 1 Implementation of Digital Controllers
- 2 Control Strategies for Industrial Processes
- 3 Programmable Logic Controllers
- 4 Real-Time Issues in Signal Transmission and Control
- 5 Communication Systems for Industrial Automation
- 6 Data Acquisition and Supervisory Control
- 7 Control of Discrete Manufacturing Processes
- 8 Intelligent Systems for Monitoring, Supervision and Control
- 9 Case Studies of Industrial Control Systems

What is an Industrial Control System (ICS)?

Definition

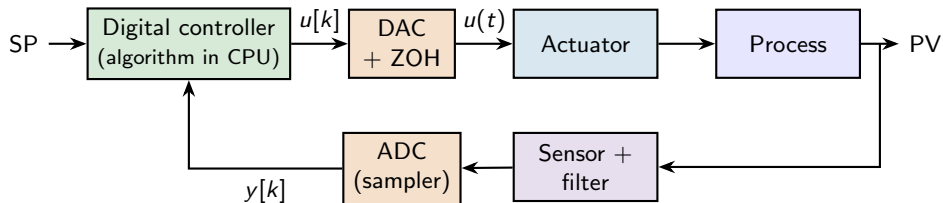
An **industrial control system** is the combination of **sensors, controllers, actuators, networks and software** that monitors and controls an industrial process — a power plant, a refinery, a factory, or a ship's engine room.

- **Continuous** processes → loop control (PID) — temperature, pressure, level, flow.
- **Discrete** processes → logic & sequence control — conveyors, machines, valves ON/OFF.
- Main families: **PLC** systems, **DCS** (distributed control system), **SCADA** (supervisory control and data acquisition).



This course: the bottom three levels.

The Digital Control Loop



- The computer only sees the process at **sampling instants** $t = kT$ (T = sample time).
- **ADC** converts the analog measurement to a number; **DAC** converts the output back.
- **Zero-order hold (ZOH)** keeps the output constant between samples $\rightarrow$ a “staircase” signal.
- Everything in between (filtering, PID, limits) is **software** — easy to change and tune.

Sampling and Quantisation

Quantisation (ADC resolution)

An n -bit ADC with full-scale range FSR has step

$$q = \frac{\text{FSR}}{2^n}$$

Example: 12-bit ADC, 0–10 V:

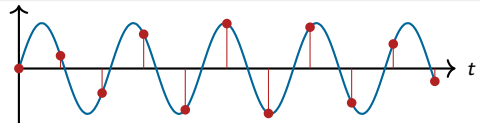
$$q = 10/4096 = \boxed{2.44 \text{ mV}}$$

Choosing the sample time T

Rule of thumb: $T \leq \tau/10$ (process time constant)
or 10–20 samples per rise time.

Aliasing & Nyquist

Sample at least **twice** the highest signal frequency ($f_s > 2f_{\max}$), and use an analog **anti-aliasing filter** before the ADC — otherwise fast noise appears as a false slow signal.



Samples (red) of a continuous signal (blue).

Discrete PID: Position and Velocity Forms

Position form

$$u[k] = K_p e[k] + K_i T \sum_{j=0}^k e[j] + \frac{K_d}{T} (e[k] - e[k-1])$$

Computes the **whole** output every sample; needs a separate anti-windup.

Velocity (incremental) form

$$\begin{aligned} \Delta u[k] &= K_p (e[k] - e[k-1]) + K_i T e[k] \\ &\quad + \frac{K_d}{T} (e[k] - 2e[k-1] + e[k-2]) \end{aligned}$$

$$u[k] = u[k-1] + \Delta u[k]$$

Computes only the **change**.

Why industry likes the velocity form

No integral windup (just clamp u between 0 and 100 %), **bumpless** MANUAL ↔ AUTO transfer, and it suits stepper-motor valves that take *increments*.

Digital Filtering of Measurements

First-order low-pass filter

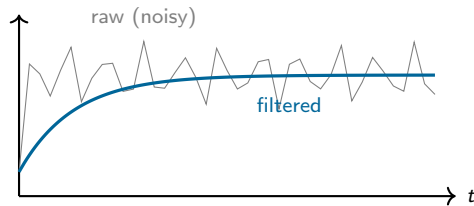
$$y_f[k] = a y_f[k-1] + (1-a)x[k], \quad a = e^{-T/\tau_f}$$

Smooths noisy measurements before the controller (especially before D action).

Example: $T = 0.1$ s, $\tau_f = 1$ s:

$$a = e^{-0.1} = 0.905$$

$$y_f[k] = 0.905 y_f[k-1] + 0.095 x[k]$$



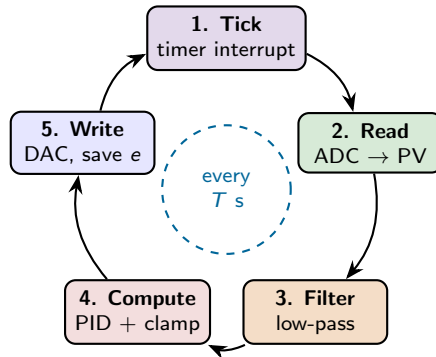
Larger τ_f = smoother, but **slower** (adds lag to the loop).

How a Digital Controller Runs: The Sample Cycle

The Tick—Read—Compute—Write cycle

Like the CPU's machine cycle, a digital controller repeats one fixed cycle, started by a **timer interrupt** every sample time T .

- 1 **Tick** — hardware timer interrupt every T (fixed, not “when ready”).
- 2 **Read** — start ADC, read PV; check for sensor fault.
- 3 **Filter** — digital low-pass filter on PV.
- 4 **Compute** — PID (velocity form), clamp 0–100 %.
- 5 **Write** — output to DAC; save $e[k]$, $e[k - 1]$; wait for next tick.



Pseudocode

- 1 START; $K_p = 2$,
 $K_i = 0.5$, $T = 1$ s
- 2 SP = 60 °C, PV
= 20 °C, $u = 0$
- 3 REPEAT every T
 - $e_0 = \text{SP} - \text{PV}$
 - du from velocity PID
 - $u = u + \text{du}$
 - clamp u to 0–100 %
 - plant responds;
shift $e_2 = e_1$, $e_1 = e_0$
- 4 STOP after 40 s

C Program (velocity-form PID)

```
#include <stdio.h>

int main(void)
{
    float Kp = 2.0, Ki = 0.5, Kd = 0.0, T = 1.0;
    float sp = 60.0, pv = 20.0, u = 0.0;
    float e0, e1 = 0.0, e2 = 0.0, du;
    int k;

    for (k = 1; k <= 40; k++) {
        e0 = sp - pv;
        du = Kp*(e0 - e1) + Ki*T*e0
            + Kd/T*(e0 - 2*e1 + e2);
        u += du;
        if (u > 100.0) u = 100.0;    /* clamp */
        if (u < 0.0) u = 0.0;
        pv += 0.1*(0.8*u + 20.0 - pv); /* plant */
        e2 = e1; e1 = e0;
        if (k % 8 == 0)
            printf("t=%2d u=%5.1f PV=%5.1f\n",
                   k, u, pv);
    }
    return 0;
}
```

Program Output

t= 8 u= 75.6 PV= 60.3
t=16 u= 50.8 PV= 63.7
t=24 u= 47.9 PV= 60.7
t=32 u= 49.5 PV= 59.8
t=40 u= 50.1 PV= 59.9

(t in s, u in %, PV in °C)

Note: the plant is a first-order heater model (0.8 °C per % output, ambient 20 °C). PV overshoots slightly, then settles at the 60 °C setpoint with about 50 % heater output. Because only du is added, clamping u is all the anti-windup needed.

Overview of Control Strategies

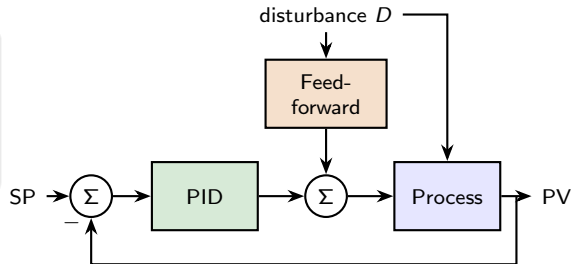
Strategy	Idea	Typical use
ON-OFF	output 0 or 100 % with a dead band	tank heaters, bilge pumps
Feedback (PID)	correct the measured error	most single loops
Feedforward	act on a <i>measured disturbance</i> before it upsets PV	boiler feedwater, heat exchangers
Cascade	outer loop sets the SP of a fast inner loop	jacket temperature, valve flow
Ratio	keep one flow a fixed ratio of another	fuel/air, blending
Split-range	one controller drives two valves	heating & cooling
Override / selector	the more critical of two controllers wins	compressor surge, limits
Advanced (MPC, adaptive)	model-based, multivariable, self-tuning	refineries, ship autopilots

Feedforward Control

Idea

Feedback waits until the disturbance has **already** changed PV. Feedforward **measures the disturbance** and corrects the output immediately. Feedback trims what feedforward misses.

- Needs a measurable disturbance and a model of its effect.
- Almost always used **with** feedback, never alone.
- Example: heat exchanger — measure inlet flow change, adjust steam at once.

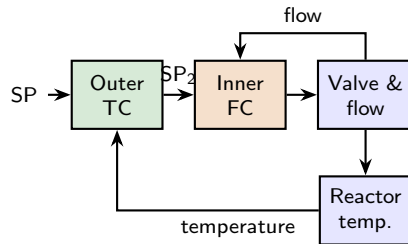


Cascade Control

Idea

Two controllers in series: the **outer (master)** controller sets the setpoint of the **inner (slave)** controller. The fast inner loop kills disturbances before they reach the slow outer variable.

- Inner loop must be **3–10× faster** than the outer.
- Tune the inner loop first, then the outer.
- Example: reactor temperature (outer) → cooling-water flow (inner).



Ratio, Split-Range and Override Control

Ratio control

Keep flow B at a fixed ratio R of flow A:

$$SP_B = R \times F_A.$$

Example: boiler fuel flow 2 t/h, air/fuel ratio 15:1

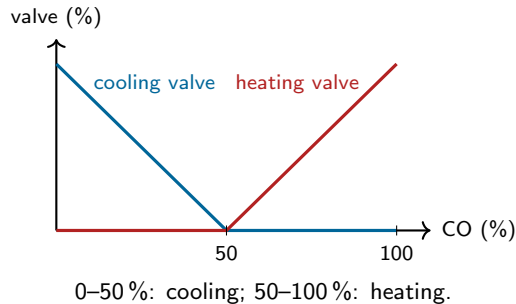
$$\Rightarrow \text{air SP} = 15 \times 2 = \boxed{30 \text{ t/h}}.$$

Override (selector) control

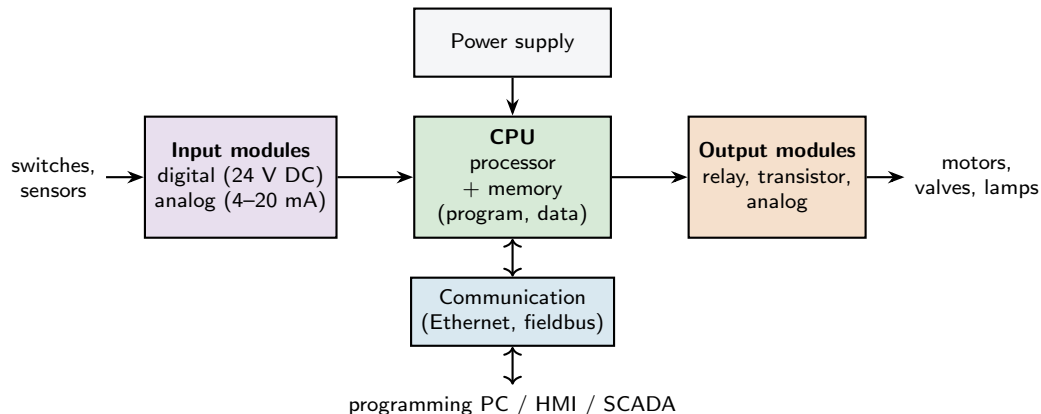
Two controllers share one valve; a **low / high selector** passes the safer output — e.g. a pressure limit overrides the flow controller.

Split-range control

One controller output drives **two** valves over different ranges of its signal.

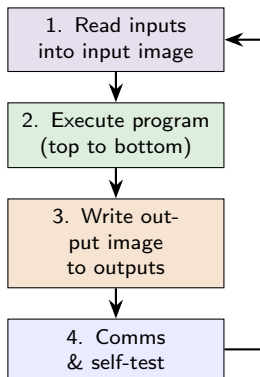


PLC Architecture



- **Rugged**: designed for heat, vibration, electrical noise — unlike an office PC.
- **Modular**: add I/O cards as the machine grows; remote I/O over a network.
- **Safety PLCs** (redundant CPUs, self-tests) for emergency shutdown systems.

The PLC Scan Cycle and Response Time



Scan time

Typically 1–50 ms, depending on program size and CPU speed.

Worst-case response time

An input can change just *after* it was read, so it is seen one scan later and acted on at the end of the next:

$$t_{\text{resp}} \approx t_{\text{in filter}} + 2 t_{\text{scan}} + t_{\text{out}}$$

Example: filter 3 ms, scan 5 ms, relay output 10 ms:

$$t_{\text{resp}} \approx 3 + 10 + 10 = \boxed{23 \text{ ms}}$$

PLC Programming Languages (IEC 61131-3)

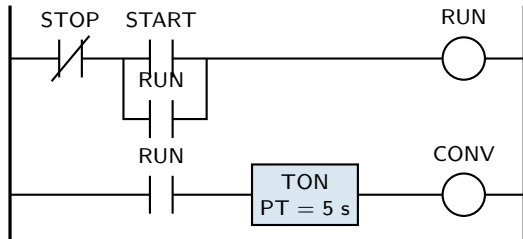
Language	Type	Looks like	Best for
Ladder Diagram (LD)	graphical	relay circuits, rungs	simple logic, electricians
Function Block Diagram (FBD)	graphical	connected blocks	process & analog control
Structured Text (ST)	text	Pascal / C	calculations, algorithms
Sequential Function Chart (SFC)	graphical	steps & transitions	sequences, batches
Instruction List (IL)	text	assembly	legacy (deprecated)

Key idea

All languages can be mixed in one project. The same logic can be written in LD or ST — choose the one that is clearest for the people who will maintain it.

Timers in Ladder Logic and Structured Text

Conveyor starts 5 s after **START** (warning horn first)



```
(* latch the RUN flag *)  
RUN := (START OR RUN) AND NOT STOP;
```

```
(* on-delay timer, 5 s *)  
T1(IN := RUN, PT := T#5s);  
CONV := T1.Q;  
HORN := RUN AND NOT T1.Q;
```

TON (on-delay): output Q turns ON when IN has been ON for the preset time PT. Counters (**CTU**, **CTD**) work the same way with pulses.

What Does “Real-Time” Mean?

Real-time system

A system whose correctness depends not only on the **result** but also on the **time** at which the result is produced. A late answer is a *wrong* answer.

Hard real-time

Missing a deadline = failure.
Airbag, ESD trip, motion control.

Firm real-time

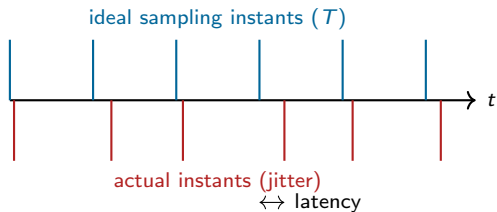
Late results are useless but tolerated occasionally.
Video frames, some sampling.

Soft real-time

Late results lose value gradually.
HMI screens, trends, reports.

- **Latency** — delay from event to reaction. **Jitter** — variation of that delay.
- **Determinism** — the worst-case time is known and guaranteed.

Latency, Jitter and their Effect on Control



Delay costs phase margin

A pure time delay τ_d adds a phase lag of $\omega\tau_d$ (rad) at frequency ω — the loop moves closer to instability.

Example: crossover frequency $\omega_c = 10$ rad/s, network + computing delay $\tau_d = 20$ ms:

$$\phi = 10 \times 0.02 = 0.2 \text{ rad} \approx \boxed{11.5^\circ}$$

If the loop had 45° phase margin, only $\approx 33.5^\circ$ remain — more overshoot.

Scheduling Real-Time Tasks

Rate-monotonic scheduling

Periodic tasks; the task with the **shortest period** gets the **highest priority**. A set of n tasks is guaranteed to meet all deadlines if

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n \left(2^{1/n} - 1 \right)$$

C_i = execution time, T_i = period.

A **watchdog timer** resets the controller (or trips to a safe state) if the program ever stops completing its cycle on time.

Task	C (ms)	T (ms)	C/T
Fast PID loop	1	10	0.10
Slow PID loops	2	20	0.10
Logic & comms	6	40	0.15
Total U			0.35

Bound for $n = 3$: $3(2^{1/3} - 1) = 0.78$

$0.35 \leq 0.78 \Rightarrow$ **schedulable** ✓

Signal Transmission Issues

Noise and interference

- **EMI** from motors, VFDs, radios, welding.
- **Ground loops** — two earth points at different potentials.
- Cable **resistance** and voltage drop on long runs.

Good practice

- Shielded twisted-pair cable; shield earthed at *one* end.
- Separate signal and power cable trays.
- Galvanic isolation (isolators, optocouplers).

Signal

Property

0–10 V

simple, but voltage drop & noise on long cables

4–20 mA

current is the same all round the loop; live zero detects broken wire

HART

digital data on top of 4–20 mA

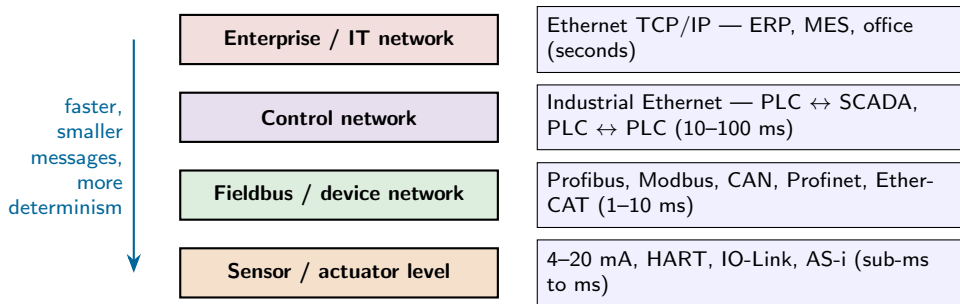
Fieldbus /
Ethernet

many values on one cable, diagnostics

Fibre optic

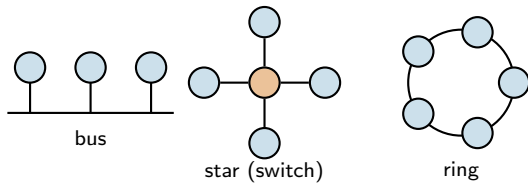
immune to EMI, long distances

Industrial Network Hierarchy



Upper levels move **large amounts of data**; lower levels move **small data very fast and predictably**.

Topologies and Medium Access



Ring topologies with **media redundancy** keep running if one cable is cut — common on ships.

Who talks when? (medium access)

Master-slave (polling) — master asks each slave in turn; deterministic (Modbus, Profibus DP).

Token passing — a token circulates; only its holder transmits.

CSMA — listen, then talk; collisions resolved by priority (CAN) or switches (Ethernet).

Time-slot (TDMA) — each node owns a fixed time window (EtherCAT, TSN).

Common Industrial Protocols

Protocol	Medium	Access	Typical use
Modbus RTU / TCP	RS-485 / Ethernet	master–slave, polling	simple, universal; meters, drives
Profibus DP	RS-485	token + master–slave	factory & process I/O
CAN / CANopen	twisted pair	CSMA with priority	machines, vehicles
NMEA 2000	CAN	CSMA with priority	ship navigation & engine data
HART	on 4–20 mA	master–slave	smart field instruments
Profinet, EtherNet/IP	Ethernet	switched, real-time classes	modern PLC systems
EtherCAT	Ethernet	on-the-fly frame processing	fast motion control

Wireless options (WirelessHART, ISA100, Wi-Fi, 5G) are used for monitoring more than for fast control.

Modbus RTU: Frame and Timing Worked Example

Request frame: “slave 1, read 2 holding registers from address 0”

Address 01	Function 03	Start address 00 00	Quantity 00 02	CRC 2 bytes
---------------	----------------	------------------------	-------------------	----------------

Problem: RS-485 at 19 200 baud, 11 bits per byte (start + 8 data + parity + stop). Request = 8 bytes, reply = 9 bytes, silent gap of 3.5 characters before each frame. How long is one poll?

$$t_{\text{req}} = \frac{8 \times 11}{19\,200} = 4.58 \text{ ms}$$

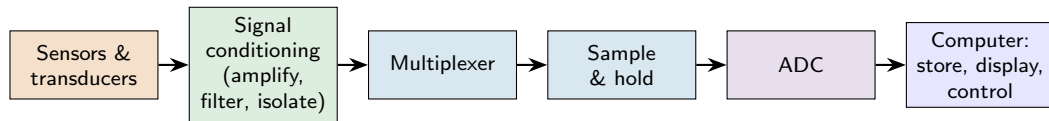
$$t_{\text{rep}} = \frac{9 \times 11}{19\,200} = 5.16 \text{ ms}$$

$$t_{\text{gap}} = \frac{3.5 \times 11}{19\,200} = 2.01 \text{ ms } (\times 2)$$

$$t_{\text{poll}} \approx \boxed{13.8 \text{ ms}} \Rightarrow \approx 72 \text{ polls/s}$$

With 10 slaves each device is refreshed only ~ 7 times per second — fine for monitoring, too slow for fast control loops.

Data Acquisition (DAQ) Systems



- One ADC is shared by many channels through the **multiplexer**.
- **Sample & hold** freezes the signal during conversion.
- Specs: channels, resolution (bits), sample rate, accuracy, isolation.

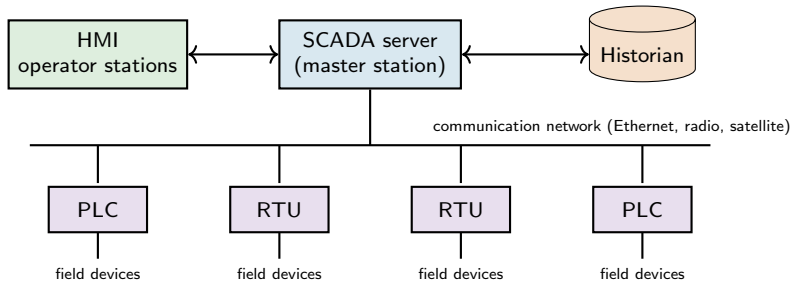
Worked example

16 temperature channels, each sampled at 100 Hz, 16-bit ADC:

$$\text{aggregate rate} = 16 \times 100 = \boxed{1600 \text{ S/s}}$$

$$\text{data rate} = 1600 \times 2 \text{ bytes} = 3.2 \text{ kB/s} \approx 276 \text{ MB/day.}$$

SCADA Architecture



- **RTU / PLC** — collect field data, run local control.
- **Master station** — polls RTUs, stores data, runs logic.
- **HMI** — mimic diagrams, alarms, trends, commands.
- **Historian** — long-term data for reports & analysis.

SCADA vs. DCS vs. PLC

Feature	PLC	DCS	SCADA
Main job	fast logic & machine control	continuous process control	supervision over wide areas
Geography	one machine / cell	one plant	pipelines, grids, fleets
Control loops	some PID	hundreds of PID loops	mostly done by RTUs / PLCs
Communication	fieldbus, Ethernet	proprietary high-reliability	WAN, radio, satellite
Typical site	packaging line	refinery, power plant	water network, ship fleet

Trend

The boundaries are blurring: modern PLCs run many PID loops, DCS vendors offer SCADA features, and all use Ethernet and the same HMI software.

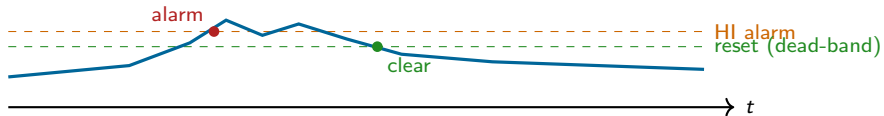
Alarm Management and HMI Design

Good alarms (ISA-18.2 / EEMUA 191 ideas)

- Every alarm needs a defined **operator action**.
- **Prioritise**: critical, high, medium, low.
- Avoid **alarm floods** — target < 1 alarm per 10 min in normal operation.
- Use dead-bands and delays to stop chattering alarms.

Good HMI screens

- Grey backgrounds; colour only for abnormal states.
- Show values **with** their limits and trends.
- Consistent navigation, overview $\rightarrow$ detail.
- Confirm critical commands.



Continuous vs. Discrete Control

Aspect	Continuous control	Discrete control
Variables	analog (temperature, flow)	binary (part present, valve open)
Changes	continuously in time	at events (switch closes, timer ends)
Model	differential equations	Boolean logic, state machines
Controller	PID, feedforward	logic & sequence control (PLC)
Example	jacket-water temperature	conveyor, pick-and-place, batch steps

Two kinds of discrete control

Logic control — outputs depend only on *present* inputs (combinational, interlocks).

Sequence control — outputs depend on the *step* the process is in (sequential, memory).

Example: press safety interlock

The press may close only if **both** hands are on the buttons (L, R), the guard G is closed, and there is no emergency stop E:

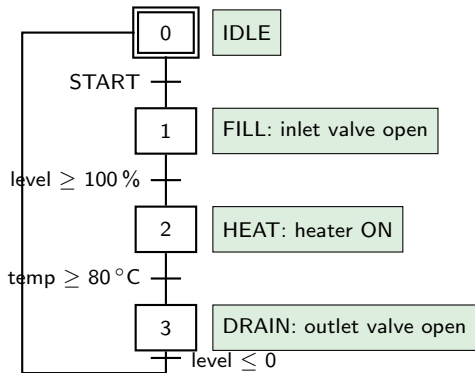
$$\text{PRESS} = L \cdot R \cdot G \cdot \bar{E}$$

- Interlocks prevent unsafe **combinations**.
- Permissives allow a start only when conditions are met.

L	R	G	E	PRESS
1	1	1	0	1
1	1	1	1	0
1	1	0	0	0
1	0	1	0	0
0	1	1	0	0

Only one row of the 16 combinations lets the press close.

Sequence Control with SFC (Grafcet)



- **Steps** (boxes) — the state the process is in; the active step's **actions** are ON.
- **Transitions** (bars) — conditions that move to the next step.
- Double box = **initial step**.
- SFC supports branches: alternative (OR) and simultaneous (AND) sequences.

Same thing in software

An SFC is a **state machine**: a variable holds the current step, and each scan checks only the transition out of that step. The C program on the next slide does exactly this.

Pseudocode

- 1 START in state FILL
- 2 REPEAT each scan until DONE
 - FILL: level += 20; if level $\geq$ 100 → HEAT
 - HEAT: temp += 15; if temp $\geq$ 80 → DRAIN
 - DRAIN: level -= 25; if level $\leq$ 0 → DONE
 - PRINT state, level, temp
- 3 STOP

C Program (tank state machine)

```
#include <stdio.h>
enum state { FILL, HEAT, DRAIN, DONE };

int main(void)
{
    const char *name[] = {"FILL", "HEAT", "DRAIN", "DONE"};
    enum state s = FILL;
    int level = 0, temp = 20, t = 0;

    while (s != DONE) {
        t++;
        if (s == FILL) { /* inlet */
            level += 20; if (level >= 100) s = HEAT;
        } else if (s == HEAT) { /* heater */
            temp += 15; if (temp >= 80) s = DRAIN;
        } else if (s == DRAIN) { /* outlet */
            level -= 25; if (level <= 0) s = DONE;
        }
        printf("t=%2d %-5s L=%3d T=%d\n",
              t, name[s], level, temp);
    }
    return 0;
}
```

Program Output

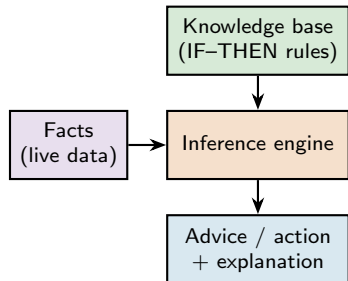
```
t= 1 FILL L= 20 T=20
t= 2 FILL L= 40 T=20
t= 3 FILL L= 60 T=20
t= 4 FILL L= 80 T=20
t= 5 HEAT L=100 T=20
t= 6 HEAT L=100 T=35
t= 7 HEAT L=100 T=50
t= 8 HEAT L=100 T=65
t= 9 DRAIN L=100 T=80
t=10 DRAIN L= 75 T=80
t=11 DRAIN L= 50 T=80
t=12 DRAIN L= 25 T=80
t=13 DONE L= 0 T=80
```

Why “intelligent”?

Classical controllers need an accurate model and fixed rules. Intelligent techniques use **expert knowledge**, **data** and **learning** to handle complex, uncertain or changing processes.

- **Expert systems** — IF–THEN rules from experienced operators.
- **Fuzzy logic** — control with vague terms (“slightly hot”).
- **Neural networks / ML** — learn patterns from data.

Task	Intelligent technique
Monitoring	soft sensors, anomaly detection
Supervision	expert systems, fault diagnosis
Control	fuzzy control, adaptive / MPC
Maintenance	predictive maintenance (ML)
Optimisation	genetic algorithms, digital twins



Example rules (engine-room pump)

IF discharge pressure LOW **AND** motor current LOW

THEN suspect **loss of suction**.

IF discharge pressure LOW **AND** motor current HIGH

THEN suspect **blocked impeller / bearing**.

IF bearing temp RISING **AND** vibration HIGH

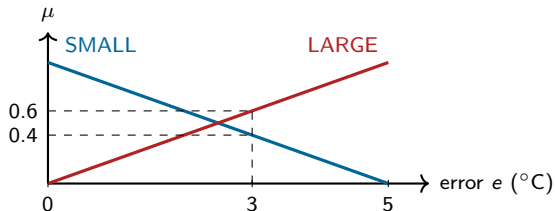
THEN advise **stop & inspect**.

Rules capture the experience of senior engineers and *explain* their conclusions.

Fuzzy Logic Control: Worked Example

Idea

Inputs belong to sets like SMALL or LARGE with a **degree** μ between 0 and 1; rules fire partially; outputs are blended.



Rules:

IF e is SMALL THEN heater = 20 %

IF e is LARGE THEN heater = 80 %

Input $e = 3^\circ\text{C}$: $\mu_S = 0.4$, $\mu_L = 0.6$

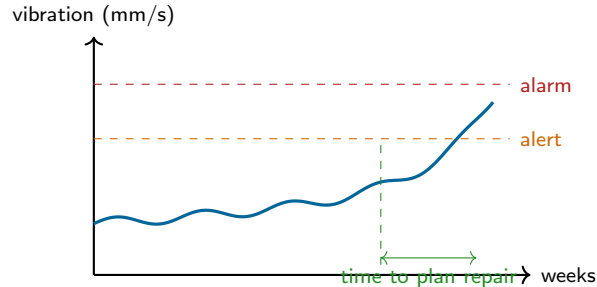
Weighted average (Sugeno):

$$u = \frac{0.4 \times 20 + 0.6 \times 80}{0.4 + 0.6} = \boxed{56\%}$$

Smooth output from simple rules — used in washing machines, cement kilns, ship stabilisers and autopilots.

Condition Monitoring and Predictive Maintenance

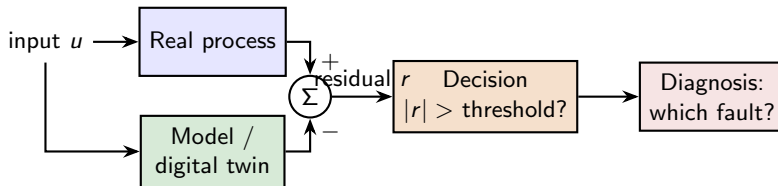
- Measure health indicators: **vibration**, temperature, oil debris, current signature, pressure drop.
- Detect **trends** early and plan repair before failure.
- ML models learn the “normal” behaviour and flag **anomalies**.
- Estimate **remaining useful life (RUL)**.



Maintenance strategies

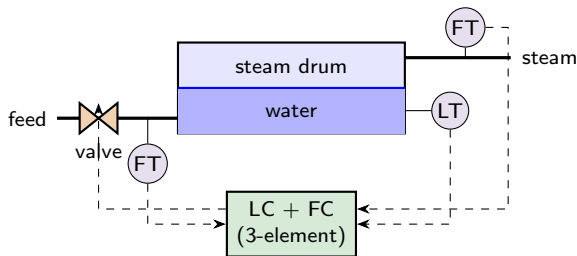
Breakdown → preventive (by calendar) →
predictive (by condition).

Fault Detection and Diagnosis



- **Residual** = measured output – model output; near zero when healthy.
- A residual above the threshold **detects** a fault; the pattern of residuals **isolates** it (sensor, actuator or process fault).
- Supervisory system then reconfigures: switch to a standby pump, use a backup sensor, or shut down safely.

Case Study 1: Boiler Drum Level — Three-Element Control

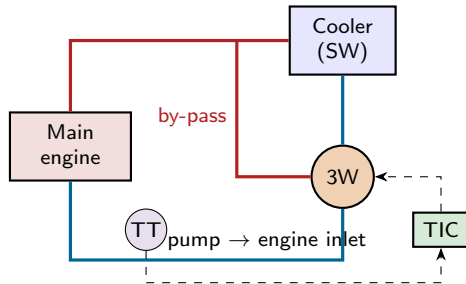


- **Element 1** — drum level (LT): the variable to control.
- **Element 2** — steam flow: **feedforward** of the load.
- **Element 3** — feedwater flow: fast **cascade** inner loop.

Why not just level control?

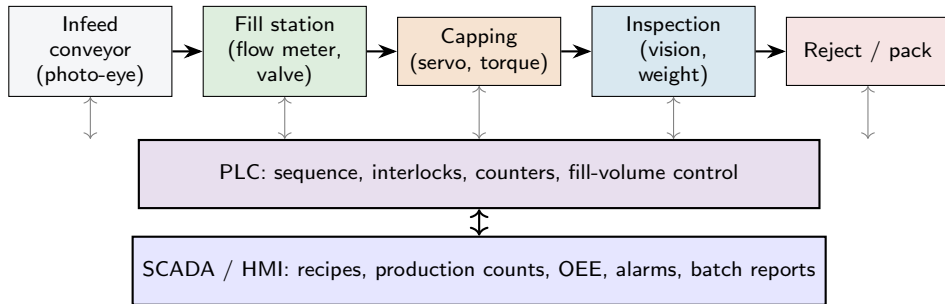
When steam demand rises, pressure falls and bubbles swell — level *rises* first (**swell**). A level-only controller would cut feedwater just when more is needed. Feedforward from steam flow avoids this.

Case Study 2: Main Engine Jacket Cooling-Water Temperature



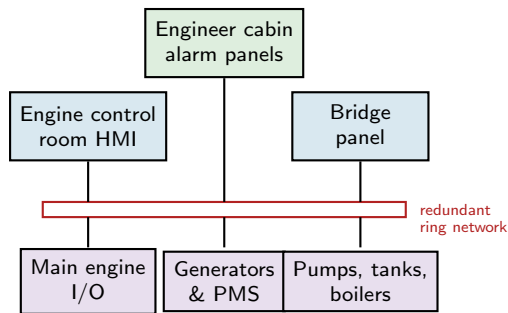
- A **three-way valve** mixes cooled and by-passed water.
- **PID** (TIC) keeps the engine inlet temperature near its setpoint (typically about 80 °C).
- Often **cascade** with outlet temperature, or **feedforward** from engine load.
- Alarms: high temperature, low pressure → slow-down / shut-down of the engine (safety system, separate from control).

Case Study 3: Bottle-Filling Line (PLC + SCADA)



- **Discrete control:** photo-eyes and counters trigger each station; a state machine per station.
- **Continuous control:** fill flow and capping torque; **intelligent** vision inspection rejects bad bottles.
- SCADA computes **OEE** = availability $\times$ performance $\times$ quality, e.g. $0.90 \times 0.95 \times 0.98 = 0.84$.

Case Study 4: Ship Alarm, Monitoring and Control System



- Thousands of I/O points: temperatures, pressures, levels, running status.
- **Distributed** I/O stations near the machinery; redundant network.
- **UMS** (unmanned machinery space): alarms routed to the duty engineer's cabin at night.
- **PMS** (power management) starts / stops generators automatically.
- Class rules require redundancy, self-monitoring and independent safety systems.

Summary & What Comes Next

Concept

- Digital controller implementation: sampling, ADC/DAC, discrete PID, filtering.
- Control strategies: feedforward, cascade, ratio, split-range, override.
- PLCs: architecture, scan cycle, IEC 61131-3.
- Real-time issues, delay, scheduling, signal transmission.
- Industrial networks and protocols.
- DAQ, SCADA, DCS, alarm management.
- Discrete control, SFC, state machines.
- Intelligent systems and case studies.

Practice makes perfect

- Run both C programs; change gains, sample time and thresholds.
- Write the tank sequence in ladder logic or ST.
- Recalculate the Modbus poll time at 9600 and 115 200 baud.
- Draw the control scheme of a system on board a ship.

Coming up

Industrial cybersecurity, safety instrumented systems (SIS) and IIoT / Industry 4.0.

Thank You

Questions?